



SOFTWARE DEVELOPMENT TRACK

Introduction to NestJS

The framework behind SEMS — the SomNOG9 Event Management System

What we'll cover



Why NestJS

What problem it solves, and how it differs from plain Express.



The TypeScript you need

Decorators, classes and constructor injection — nothing more.



Your first project

The Nest CLI, project layout and the dev server.



Modules, controllers, providers

The three building blocks every Nest app is made of.



Validation, lifecycle, guards

DTOs, pipes, the request pipeline and auth.



Prisma & microservices

How today maps onto the SEMS services we are about to build.

DEFINITION

So what is NestJS?

A **progressive Node.js framework** for building server-side applications — opinionated where Express leaves you to decide.



Written in TypeScript

Types, decorators and interfaces are first-class, not bolted on.



Runs on Express or Fastify

It is not a new HTTP server — it wraps one you already know.



Architecture decided for you

Modules, controllers and providers, in every project.



One model, many transports

REST, GraphQL, WebSockets and message queues share the same code.

MOTIVATION

Express gives you freedom. Nest gives you conventions.

Concern	Plain Express	NestJS
Project structure	You invent one per project	Same structure in every project
Type safety	Optional, bolted on later	TypeScript from the first file
Wiring dependencies	Manual require and globals	Built-in DI container
Input validation	Pick and wire a library	DTOs + ValidationPipe out of the box
Testing	You build the harness	Test module that swaps real providers for fakes
Microservices	Another library, another pattern	Same decorators, different transport

Express is a library. Nest is a **set of decisions** — which is exactly what a team of five services and many contributors needs.

PREREQUISITES

The TypeScript you actually need

```
// A decorator is just a function that attaches metadata
@Injectable()
export class EventsService {
  // 'private readonly' in the constructor declares
  // the property AND assigns it in one line
  constructor(private readonly prisma: PrismaService) {}

  findAll(): Promise<Event[]> {
    return this.prisma.event.findMany();
  }
}
```

Decorators

@Injectable, **@Controller**, **@Get** — Nest reads them at startup to learn what your classes are.

Parameter properties

A constructor parameter marked **private readonly** becomes a class property. This is how you receive dependencies.

Types survive

Types are erased at runtime, so Nest relies on **emitDecoratorMetadata** to know which class to inject.

SETUP

Five commands to a running server

```
# 1. install the CLI once
npm i -g @nestjs/cli

# 2. create a project (it asks: CommonJS or ESM?)
nest new sems-events
cd sems-events
npm run start:dev          # http://localhost:3000

# 3. generate the pieces
nest g module      events
nest g controller events
nest g service     events
nest g resource    registrations  # all three + CRUD
```

Check with **node -v** before you start — this trips people up first.

ESM projects default to Vitest and oxlint; CommonJS keeps Jest and ESLint. Either is fine for the lab.

Same generators, run inside the Nx monorepo — the files land under **apps/<service>/src**.

LAYOUT

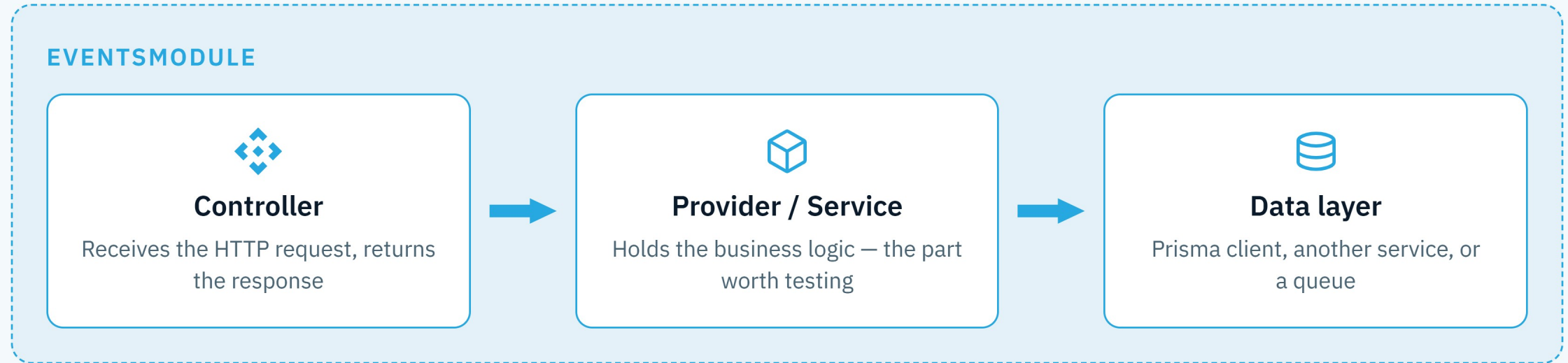
One folder per feature

```
src/
├─ main.ts           # bootstrap: creates the app
├─ app.module.ts     # the root module
├─ events/
│   ├─ events.module.ts # wires the feature together
│   ├─ events.controller.ts # HTTP routes
│   ├─ events.service.ts # business logic
│   └─ dto/
│       ├─ create-event.dto.ts
│       └─ update-event.dto.ts
└─ entities/event.entity.ts
```

```
// main.ts – every Nest app starts here
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.useGlobalPipes(new ValidationPipe());
  await app.listen(3000);
}
bootstrap();
```

One folder per feature. Every feature folder holds the same four kinds of file, so any developer can open any folder and know what is where.

Three building blocks, one box around them



A **module** is the box. A **controller** is the door. A **provider** is everything that actually does work.

Modules — the unit of organisation

```
@Module({
  imports:      [PrismaModule, NotificationsModule],
  controllers:  [EventsController],
  providers:    [EventsService],
  exports:      [EventsService],
})
export class EventsModule {}
```

imports

Other modules whose exported providers this one may use.

controllers

The classes that handle incoming requests for this feature.

providers

Classes Nest may instantiate and inject here.

exports

What other modules get when they import this one.

Controllers — routes, and nothing else

```
@Controller('events') // → /events
export class EventsController {
  constructor(private readonly events: EventsService) {}

  @Get() // GET /events
  findAll(@Query('year') year?: string) {
    return this.events.findAll(year);
  }

  @Get('/:id') // GET /events/42
  findOne(@Param('id', ParseIntPipe) id: number) {
    return this.events.findOne(id);
  }

  @Post() // POST /events
  create(@Body() dto: CreateEventDto) {
    return this.events.create(dto);
  }
}
```

Routing only

A controller maps a URL to a method. No business logic, no SQL — ever.

Parameter decorators

@Body, **@Param**, **@Query**, **@Headers** pull pieces out of the request for you.

Return a value

Return an object or a promise and Nest serialises it to JSON with status 200 (201 for POST).

Providers — where the work happens

```
@Injectable()
export class EventsService {
  constructor(private readonly prisma: PrismaService) {}

  findAll(year?: string) {
    return this.prisma.event.findMany({
      where: year ? { year: Number(year) } : undefined,
      orderBy: { startsAt: 'asc' },
    });
  }

  async findOne(id: number) {
    const event = await this.prisma.event.findUnique({ where: { id } });
    if (!event) throw new NotFoundException(`Event ${id} not found`);
    return event;
  }
}
```

@Injectable()

Marks the class as something Nest can construct and inject.

This is where logic lives

Rules, calculations, calls to other services — all here, all unit-testable.

Throw, don't format

NotFoundException becomes a proper JSON 404. Nest's exception filter does the work.

THE IDEA THAT TIES IT TOGETHER

Dependency injection in three steps

You never call **new**. You declare what you need, and Nest provides it.

1

You mark the class

@Injectable() on `EventsService`.

2

You list it in a module

providers: [EventsService].

3

Nest hands it to you

It reads the constructor's types and passes in a single shared instance.



Because you never wrote **new EventsService()**, a test can hand it a fake Prisma client instead of a real database — no mocking framework, no patching.

INPUT

DTOs describe the shape; pipes enforce it

```
// create-event.dto.ts
export class CreateEventDto {
  @IsString() @MinLength(3)
  title: string;

  @IsISO8601()
  startsAt: string;

  @IsInt() @Min(1)
  capacity: number;
}
```

```
// main.ts
app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,
    transform: true,
  }),
);
```

Rejected before your code runs

A bad body returns 400 with a list of what failed — the service is never called.

NestJS 12 adds Standard Schema

Zod, Valibot or ArkType via a **schema** option on the route decorator, if you prefer them to decorators.

What happens between request and response



Stage	Question it answers	Typical use in SEMS
Middleware	Raw request and response	Logging, CORS, request id
Guards	May this caller proceed?	JWT check, role check
Interceptors	Wrap the call, both sides	Timing, caching, response shape
Pipes	Validate and transform input	ValidationPipe, ParseIntPipe
Route handler	Your controller method	Calls the service
Exception filters	Turn a thrown error into a response	NotFoundException → 404

Guards decide who gets in

```
@Injectable()
export class JwtAuthGuard implements CanActivate {
  constructor(private readonly jwt: JwtService) {}

  canActivate(ctx: ExecutionContext): boolean {
    const req = ctx.switchToHttp().getRequest();
    const token = req.headers.authorization?.split(' ')[1];
    if (!token) throw new UnauthorizedException();
    req.user = this.jwt.verify(token);
    return true;
  }
}

@UseGuards(JwtAuthGuard)
@Post()
create(@Body() dto: CreateEventDto) { /* ... */ }
```

A guard returns true or false

True lets the request through; false or a thrown exception stops it with 401 or 403.

Apply it anywhere

@UseGuards on a method, on a controller, or globally in **main.ts**.

In SEMS

This guard belongs to the Auth service and ships to the other services as a shared library in **libs/**.

Prisma plugs in as a provider

```
@Injectable()
export class PrismaService extends PrismaClient
  implements OnModuleInit {
  async onModuleInit() {
    await this.$connect();
  }
}

@Module({
  providers: [PrismaService],
  exports:   [PrismaService], // other modules may inject it
})
export class PrismaModule {}
```

Prisma is just a provider

Wrap the generated client in an `@Injectable` class and Nest injects it like anything else.

Lifecycle hooks

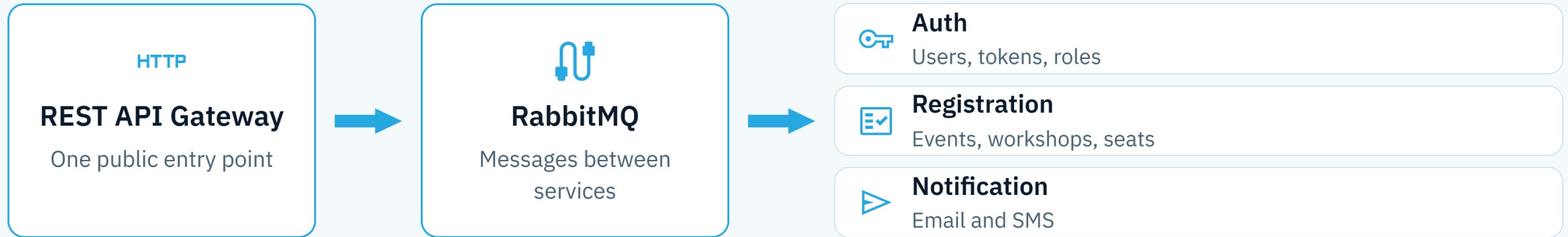
onModuleInit and **onModuleDestroy** let you open and close the connection with the app.

In SEMS

One Postgres instance, one schema per service — each service owns its own Prisma schema file.

WHERE THIS IS GOING

Today's building blocks, five services wide



```
@MessagePattern('registration.created') // consumer
handle(@Payload() data: RegistrationCreated) { /* ... */ }

this.client.emit('registration.created', payload); // producer
```

Same modules, same controllers, same providers — only the transport changes.

YOUR TURN

Hands-on lab — 30 minutes



Scaffold

nest new sems-events, then **nest g resource events**.



Model it

Write **CreateEventDto** with title, startsAt and capacity, and turn on the ValidationPipe.



Fake the data

An in-memory array inside EventsService — no database today.



Prove it

curl a good POST, then a bad one, and read the 400 you get back.



Next session: the Nx monorepo skeleton and the first Prisma schema. · Docs: docs.nestjs.com